Full Abstract – StreamSense

Aufgabe 2: Near-Real-Time-Backend für eine datenintensive Applikation

Zielsetzung und fachliche Einordnung

StreamSense ist ein lokal ausführbares Backend für simulierte Raumluftdaten. Im Mittelpunkt steht die Dateninfrastruktur: Ereignisse werden fortlaufend aufgenommen, validiert, gespeichert, verdichtet und über eine API bereitgestellt. Das Studienskript ordnet Stream Processing als Beinahe-Echtzeit-Verarbeitung zwischen Online- und Batch-Systemen ein. Deshalb beschreibe ich die technische Lösung als Near-Real-Time-Backend, auch wenn die Aufgabenstellung den Oberbegriff Real-Time-Backend verwendet (IU Internationale Hochschule, 2026, S. 36-37).

Architektur und Datenfluss

Ein Python-Simulator erzeugt Temperatur-, Feuchte-, CO_2-, Batterie- und Statuswerte mit Ereignis- und Emissionszeit. Ein konfigurierbarer Anteil erhält absichtlich eine ältere Ereigniszeit, damit verspätete Daten praktisch geprüft werden können. Für den Lasttest werden genau eine Million Ereignisse erzeugt. Kafka übernimmt die kontinuierliche Aufnahme. Das Topic besitzt sechs Partitionen; die Raum-ID dient als Schlüssel. Der Persister prüft JSON-Schema und Pydantic-Modell. Gültige Ereignisse werden in PostgreSQL gespeichert. Ungültige Nachrichten gehen in das Dead-Letter-Topic. Herkunft, Fehlergrund und Payload-Hash werden zusätzlich in einem Rejection-Audit dokumentiert.

Zustellung, Speicherung und Zeitfenster

Die Verarbeitung arbeitet nach dem At-least-once-Prinzip. Kafka-Offsets werden erst nach erfolgreichem Datenbank-Commit bestätigt. Ereignis-ID sowie Topic, Partition und Offset verhindern doppelte Rohdatensätze bei Wiederholungen. Die Auswertung ist bewusst hybrid: Der Aggregator liest alle fünf Sekunden einen begrenzten Ereigniszeitraum aus PostgreSQL und berechnet Ein-Minuten-Tumbling-Windows sowie gleitende Fünf-Minuten-Fenster. Watermark, Lookback und Upserts berücksichtigen verspätete Ereignisse und machen Neuberechnungen reproduzierbar. Der Vorteil ist die einfache lokale Nachvollziehbarkeit; der Nachteil sind wiederholte Datenbankscans bei hohem Durchsatz.

Qualitätseigenschaften, Sicherheit und Governance

Reliability unterstützen persistente Volumes, Neustartregeln, Idempotenz, DLQ, Rejection-Audit, Checkpoints und eine Advisory Lock. Ein Broker und eine Datenbankinstanz bieten jedoch keine vollständige Hochverfügbarkeit. Skalierbarkeit entsteht durch Kafka-Partitionen und Consumer Groups; der Persister kann horizontal vervielfältigt werden. Grenzen bleiben der einzelne Broker, PostgreSQL und der jeweils aktiv schreibende Aggregator. Wartbarkeit sichern getrennte Container, gemeinsame Modelle, zentrale Konfiguration, strukturierte Logs, Prometheus, Unit-Tests, CI und dokumentierte Schnittstellen. Damit werden Operabilität, Einfachheit und Evolvierbarkeit aufgegriffen (IU Internationale Hochschule, 2026, S. 30-32).

Persister, Aggregator und API teilen sich im lokalen Demonstrator eine PostgreSQL-Datenbank. Diese stärkere Kopplung ist ein bewusster Kompromiss. Getrennte Rollen begrenzen die Rechte: Writer für Ingestion, Aggregator für Fenster und Retention, Reader für die API. Die API kann nicht schreiben. App-Container laufen schreibgeschützt und ohne zusätzliche Linux-Capabilities. Verarbeitet werden synthetische Daten; bei einem realen Einsatz wären mögliche Rückschlüsse aus Raum- und Zeitmustern neu zu bewerten. Governance umfasst UTC, Einheiten, Schema-Versionierung, Herkunft, Qualitätsregeln, Retention sowie Data Owner, Data Steward und Kontrollmechanismen (IU Internationale Hochschule, 2026, S. 104-108).

Validierung und persönliche Reflexion

Die technische Validierung umfasst zehn erfolgreiche Unit-Tests für Ereignismodell, Generator, reproduzierbare IDs, verspätete Ereignisse, Fenstergrenzen und JSON-Schema. Compose-Datei, Datenbankschema und Rollenaufteilung wurden zusätzlich strukturell geprüft. Das Repository enthält einen automatisierten Smoke-Test sowie einen Lasttest, der exakt eine Million Ereignisse erzeugt. Laufzeit und Durchsatz sind hardwareabhängig und können deshalb über das beigefügte automatische Testprotokoll getrennt dokumentiert werden.

Rückblickend war die größte Herausforderung, die Aufgaben von Kafka, Persister, Datenbank und Aggregator sauber zu trennen. Besonders wichtig waren für mich Offset-Management, idempotente Speicherung und der Umgang mit verspäteten Ereignissen. Ich habe gelernt, große Anforderungen in kleine Prüfschritte zu zerlegen, Annahmen zu dokumentieren und Fehler systematisch einzugrenzen. Als Soft Skills nehme ich Priorisierung, Ausdauer und nachvollziehbare Dokumentation mit. Beim nächsten Projekt würde ich den vollständigen Container- und Lasttest früher einplanen.

Ergänzende Batch-Pipeline

Ein Backfill ruft dieselbe idempotente Aggregationsfunktion für frei gewählte historische Zeiträume auf. Sehr verspätete Ereignisse oder geänderte Regeln können dadurch ohne zweite Fachlogik verarbeitet werden. Für größere Datenmengen wären Staging-Bereich, Orchestrierung, getrennte Checkpoints und eigene Laufprotokolle sinnvoll.